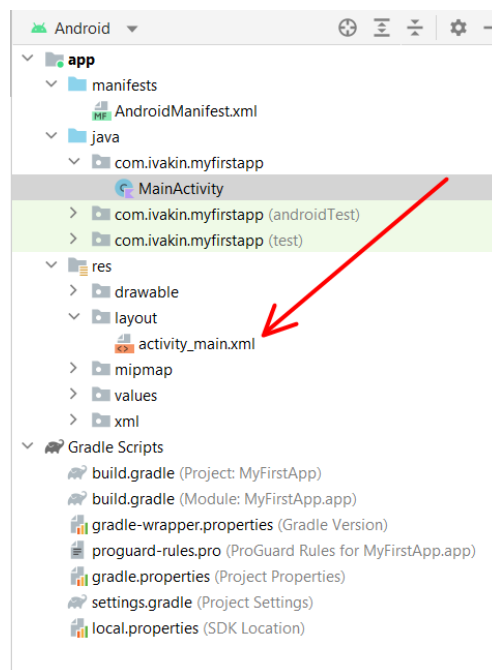


Редактор xml

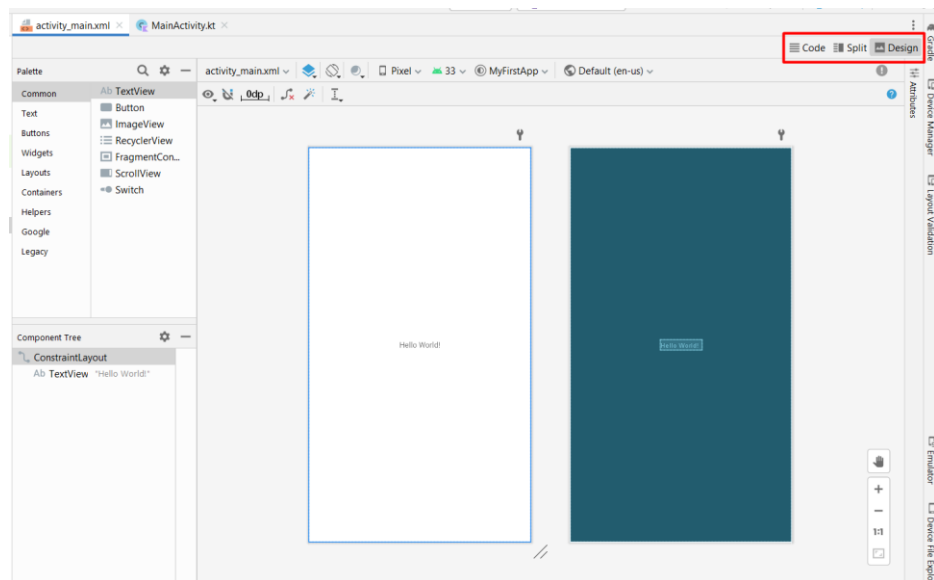
Для редактирования xml-файлов в Android Studio встроен удобный инструмент. Он позволяет не только редактировать код xml-файлов, но и видеть изменения в режиме реального времени с помощью визуального представления. В этом же визуальном представлении можно редактировать xml-файлы.

Чтобы посмотреть работу этого редактора, откройте файл `activity_main.xml`, сгенерированный Android Studio. Кстати, если ваш проект уже разросся и искать файлы неудобно, то нажмите клавишу Shift быстро два раза, и откроется окно поиска по файлам проекта. Расположение xml файла можно увидеть на скриншоте:



У файла разметки экрана приложения есть 3 варианта отображения

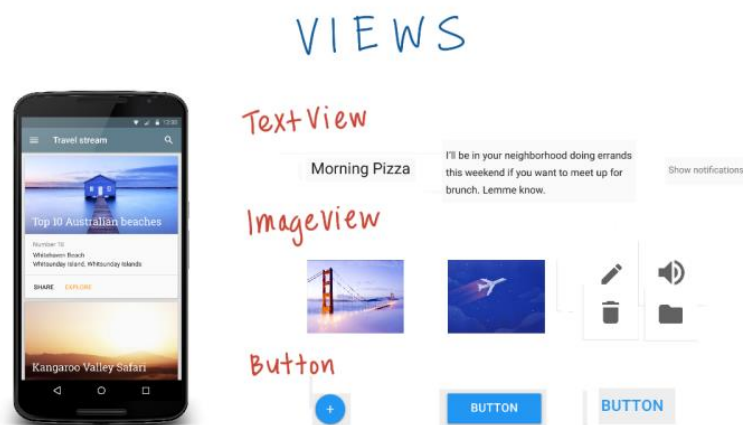
1. Design – отображает то, что уже запрограммировано
2. Code – режим, в котором мы будем создавать разметку
3. Split – режим, который отображает оба режима, наиболее удобный при построении разметки.



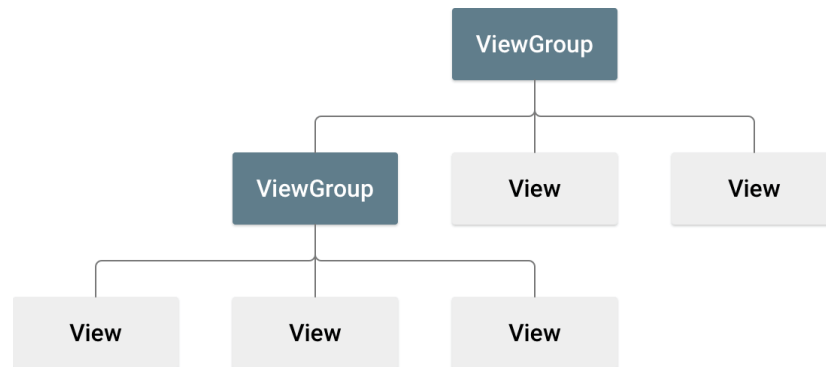
Сразу после создания проекта xml файл разметки содержит Layout, которые описывает расположение элементов внутри него и единственное View, которое называется TextView и предназначен для отображения текста “Hello World!”

Пользовательский интерфейс

Любое приложение отображает пользователю графический интерфейс. На картинке ниже вы можете увидеть примеры текстовых элементов (TextView), кнопок (Button), картинок (ImageView).



Пользовательский интерфейс представлен layout файлом, который создаётся, используя язык разметки XML. Макет определяет структуру пользовательского интерфейса экрана. Все элементы макета построены с использованием иерархии объектов View и ViewGroup. View – компонент, который пользователь может видеть и взаимодействовать с ним. В то время как ViewGroup является невидимым контейнером, который определяет структуру макета, как показано на рисунке.



Объекты View обычно называют «виджетами». Они могут быть представлены одним из классов-наследников, таких как Button, TextView, ImageView. Объекты ViewGroup обычно называют «контейнерами». Они определяют, как именно будут располагаться элементы внутри экрана.

Вы можете работать с макетами двумя способами:

1. Объявлять элементы пользовательского интерфейса в XML коде. Android предоставляет простой XML файл, в котором можно добавлять разные View и ViewGroup вручную. Вы также можете использовать редактор макетов Android Studio, чтобы создать свой XML-макет, не заглядывая в XML код.
2. Создавать элементы макета во время выполнения программы из Java кода. Приложение может создавать объекты View и ViewGroup (и управлять их свойствами) программно.

В этом уроке мы подробнее будем рассматривать именно первый вариант. Для ознакомления с синтаксисом языка XML мы будем вносить изменения в файл `activity_main.xml`.

XML синтаксис

XML – язык разметки, определяющий теги (элементы) и их атрибуты. Язык очень похож на HTML. Давайте разберём на примере.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
</LinearLayout>
```

Тег ограничивается скобками `< >`. Название тега в данном случае – `LinearLayout`. Всё, что находится между скобками – называется атрибутами. В примере два атрибута:

`android:layout_width="match_parent"`

`android:layout_height="match_parent"`

Атрибуты состоят из названия и значения, которые разделены символом `=`, причём значение атрибута всегда пишется в кавычках. У атрибута `android:layout_width="match_parent"` название – `android:layout_width`, а значение – `match_parent`.

После открытия тега, его обязательно надо закрывать. Это можно сделать используя конструкцию `</ИмяТега>` (в примере – `</LinearLayout>`).

У элемента могут быть вложенные элементы:

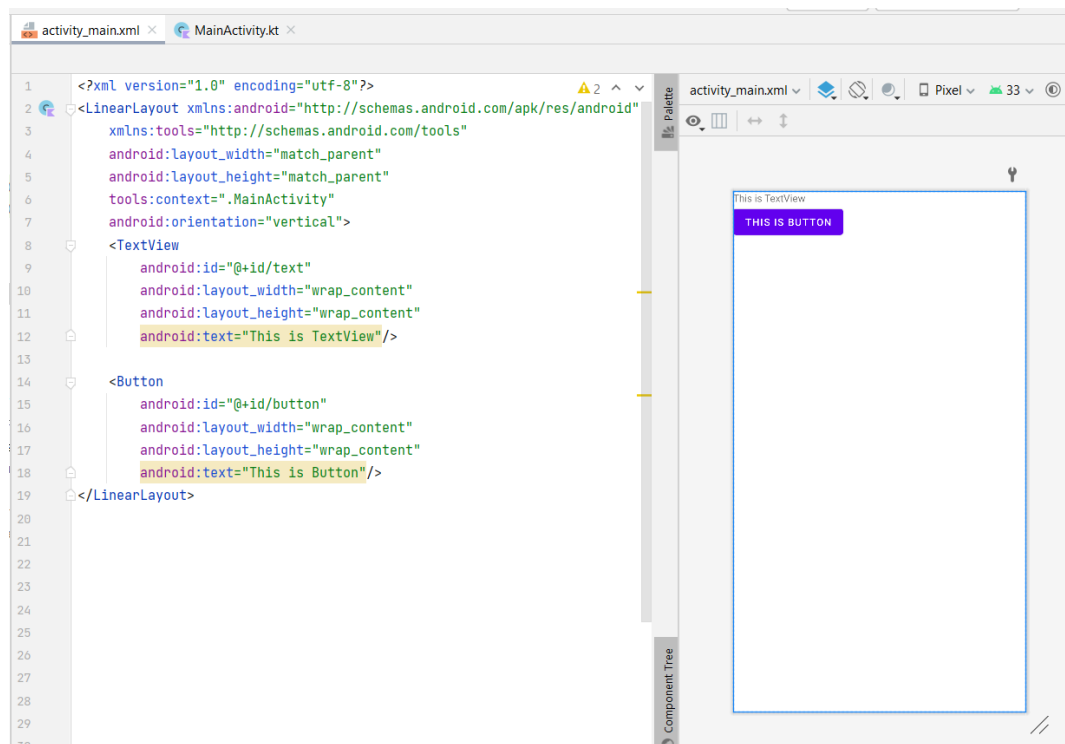
```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content">
    </TextView>
</LinearLayout>
```

Если у какого-то тега нет вложенных элементов, то лучше сократить закрывающийся тег, используя конструкцию `<ИмяТега />`. Обратите внимание на тег `TextView`:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
</LinearLayout>
```

Пример макета

Layout должен содержать только один корневой элемент, который должен быть объектом `View` или `ViewGroup` (обычно используют `ViewGroup`). После того, как вы определили корневой элемент, вы можете добавить дополнительные объекты в качестве дочерних элементов, чтобы постепенно создавать пользовательский интерфейс. Давайте изменим файл `activity_main.xml`. Например, вот код XML-макета, который использует контейнер `LinearLayout` в качестве корневого элемента и два виджета внутри него: `TextView` и `Button`.



Основные компоненты и их атрибуты

Каждый компонент обладает обязательными атрибутами, которыми являются `android:layout_width` и `android:layout_height`. Они задают ширину и высоту компонента. Мы можем указывать для них абсолютные значения, а можем использовать константы. Разберем подробнее эти возможности.

Абсолютные значения:

Используются следующие единицы измерения (ЕИ):

1. `dp` или `dip` - Density-independent Pixels. Абстрактная ЕИ, позволяющая приложениям выглядеть одинаково на различных экранах и разрешениях.
2. `sp` - Scale-independent Pixels. То же, что и `dp`, только используется для размеров шрифта в `View` элементах
3. `pt` - 1/72 дюйма, определяется по физическому размеру экрана. Эта ЕИ из типографии.
4. `px` – пиксель, не рекомендуется использовать т.к. на разных экранах приложение будет выглядеть по-разному.
5. `mm` – миллиметр, определяется по физическому размеру экрана
6. `in` – дюйм, определяется по физическому размеру экрана

Константы:

1. `match_parent` – означает, что элемент займет всю доступную ему в родительском элементе ширину/высоту.
2. `wrap_content` – ширина/высота элемента будет определяться его содержимым

Рассмотрим некоторые виджеты, которые нам потребуются:

1. `TextView`

Компонент `TextView` предназначен для отображения текста без возможности редактирования его пользователем, что видно из его названия (`Text` - текст, `view` - просмотр).

`TextView` - один из самых используемых компонентов. С его помощью пользователю удобнее ориентироваться в программе. По сути, это как таблички: Руками не трогать, По газону не ходить, Вход с собаками воспрещен, Часы работы с 9.00 до 18.00 и т.д., и служит для представления пользователю описательного текста.

Основные атрибуты:

Для отображения текста в `TextView` в файле разметки используется атрибут `android:text`.

`android:textsize`. Размер текста. При установке размера текста используется несколько единиц измерения: `px` (пиксели), `dp`, `sp`, `in` (дюймы), `pt`, `mm`. Для текстов рекомендуется использовать `sp`: `android:textSize="48sp"`, аналог - метод `setTextSize()`

`android:textstyle`. Стиль текста. Используются константы: `normal`, `bold`, `italic`. Например, `android:textStyle="bold"` выводит текст жирным

`android:textcolor`. Цвет текста. Используются четыре формата в шестнадцатеричной кодировке: `#RGB`; `#ARGB`; `#RRGGBB`; `#AARRGGBB`, где `R`, `G`, `B` — соответствующий цвет, `A` — прозрачность (alpha-канал). Значение `A`, установленное в `0`, означает прозрачность 100%.

2. `EditText`

Компонент `EditText` — это текстовое поле для пользовательского ввода, которое используется, если необходимо редактирование текста. Следует заметить, что `EditText` является наследником `TextView`.

Основные атрибуты такие же, как и у `TextView`, однако есть некоторые, свойственные именно этому виджету:

`android:hint`. Он позволяет задать текст, который будет отображаться в качестве подсказки, если элемент `EditText` пуст.

`android:inputType`. Позволяет задать клавиатуру для ввода. В частности, среди его значений можно выделить следующие:

- `text`: обычная клавиатура для ввода однострочного текста
- `textMultiLine`: многострочное текстовое поле
- `textEmailAddress`: обычная клавиатура, на которой присутствует символ `@`, ориентирована на ввод email
- `textUri`: обычная клавиатура, на которой присутствует символ `/`, ориентирована на ввод интернет-адресов
- `textPassword`: клавиатура для ввода пароля
- `textCapWords`: при вводе первый введенный символ слова представляет заглавную букву, остальные - строчные
- `number`: числовая клавиатура
- `phone`: клавиатура в стиле обычного телефона
- `date`: клавиатура для ввода даты
- `time`: клавиатура для ввода времени

datetime: клавиатура для ввода даты и времени

3. Button

Одним из часто используемых элементов являются кнопки, которые представлены классом `android.widget.Button`. Ключевой особенностью кнопок является возможность взаимодействия с пользователем через нажатия.

Некоторые ключевые атрибуты, которые можно задать у кнопок:

`text`: задает текст на кнопке

`textColor`: задает цвет текста на кнопке

`background`: задает фоновый цвет кнопки

`textAllCaps`: при значении `true` устанавливает текст в верхнем регистре. По умолчанию как раз и применяется значение `true`

4. CheckBox

Элементы `CheckBox` представляют собой флажки, которые могут находиться в отмеченном и неотмеченном состоянии. Флажки позволяют производить множественный выбор из нескольких значений.

Теперь рассмотрим 2 наиболее популярных `ViewGroup`: `ConstraintLayout` и `LinearLayout`

1. ConstraintLayout

`ConstraintLayout` представляет контейнер, который позволяет создавать гибкие и масштабируемые визуальные интерфейсы. Для позиционирования элемента внутри `ConstraintLayout` необходимо указать ограничения (`constraints`). Есть несколько типов ограничений. В частности, для установки позиции относительно определенного элемента используются следующие ограничения:

`layout_constraintLeft_toLeftOf`: левая граница позиционируется относительно левой границы другого элемента

`layout_constraintLeft_toRightOf`: левая граница позиционируется относительно правой границы другого элемента

`layout_constraintRight_toLeftOf`: правая граница позиционируется относительно левой границы другого элемента

`layout_constraintRight_toRightOf`: правая граница позиционируется относительно правой границы другого элемента

`layout_constraintTop_toTopOf`: верхняя граница позиционируется относительно верхней границы другого элемента

`layout_constraintTop_toBottomOf`: верхняя граница позиционируется относительно нижней границы другого элемента

`layout_constraintBottom_toBottomOf`: нижняя граница позиционируется относительно нижней границы другого элемента

`layout_constraintBottom_toTopOf`: нижняя граница позиционируется относительно верхней границы другого элемента

`layout_constraintBaseline_toBaselineOf`: базовая линия позиционируется относительно базовой линии другого элемента

`layout_constraintStart_toEndOf`: элемент начинается там, где завершается другой элемент

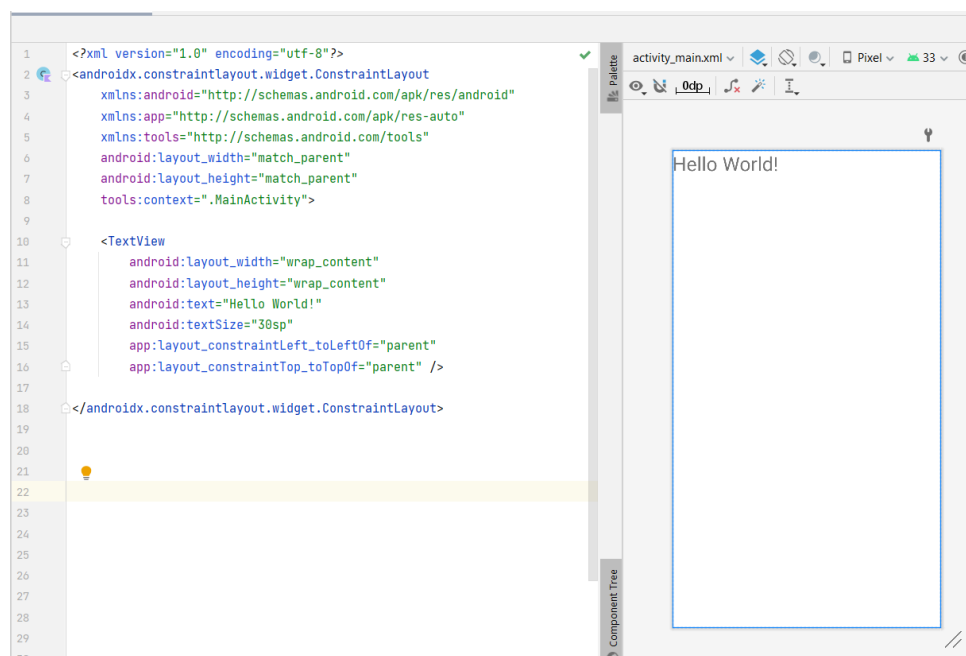
layout_constraintStart_toStartOf: элемент начинается там, где начинается другой элемент
layout_constraintEnd_toStartOf: элемент завершается там, где начинается другой элемент
layout_constraintEnd_toEndOf: элемент завершается там, где завершается другой элемент

Возможно, по поводу четырех последних свойств возникло некоторое непонимание, что подразумевается под началом или завершением элемента. Дело в том, что некоторые языки (например, арабский или фарси) имеют правостороннюю ориентацию, то есть символы идут справа налево, а не слева направо, как в европейских языках. И в зависимости от текущей ориентации - левосторонняя или правосторонняя - будет изменяться то, где именно начало, а где завершение элемента. Например, при левосторонней ориентации начало - слева, а завершение - справа, поэтому атрибут layout_constraintStart_toEndOf фактически будет аналогичен атрибуту layout_constraintLeft_toRightOf. А при правосторонней ориентации - атрибуту layout_constraintRight_toLeftOf, так как начало справа, а завершение - слева

Каждое ограничение устанавливает позиционирование элемента либо по горизонтали, либо по вертикали. И для определения позиции элемента в ConstraintLayout необходимо указать как минимум одно ограничение по горизонтали и одно ограничение по вертикали.

Позиционирования может производиться относительно границ самого контейнера ConstraintLayout (в этом случае ограничение имеет значение parent), либо же относительно любого другого элемента внутри ConstraintLayout, тогда в качестве значения ограничения указывается id этого элемента.

Простейший пример:



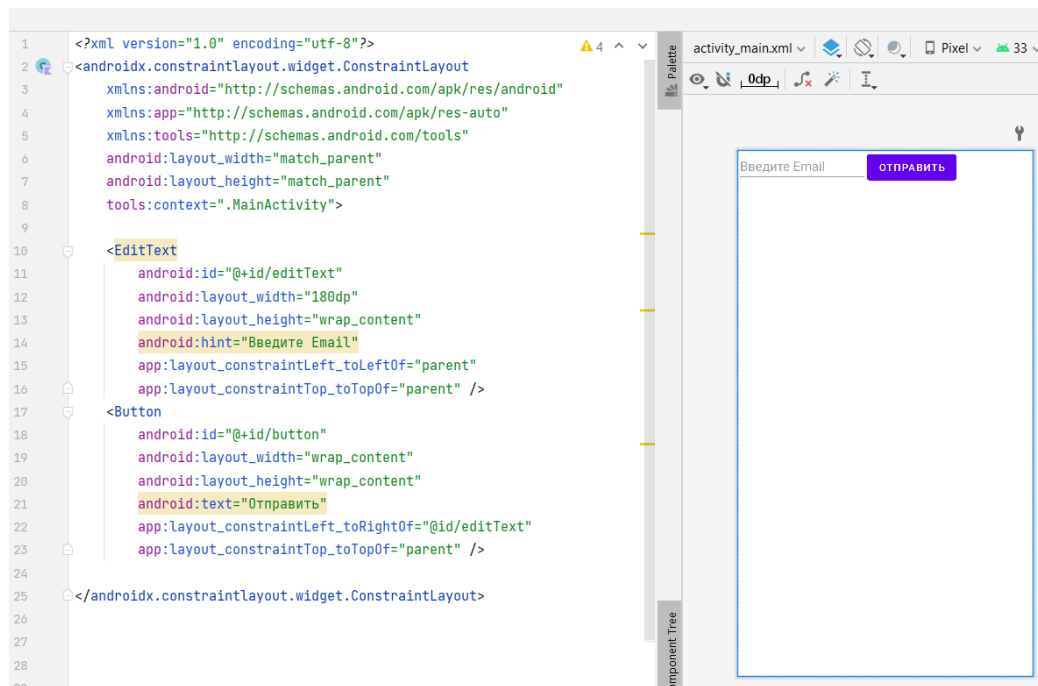
В данном случае у элемента TextView установлено два ограничения: одно по горизонтальной линии (app:layout_constraintLeft_toLeftOf="parent"), второе - по вертикальной линии (app:layout_constraintTop_toTopOf="parent"). Оба ограничения устанавливаются относительно контейнера ConstraintLayout, поэтому они принимают значение parent, то есть ConstraintLayout.

Ограничение `app:layout_constraintLeft_toLeftOf="parent"` устанавливает левую границу `TextView` у левой границы контейнера.

Ограничение `app:layout_constraintTop_toTopOf="parent"` устанавливает верхнюю границу `TextView` у верхней границы контейнера.

В итоге `TextView` будет располагаться в верхнем левом углу контейнера.

Если необходимо установить ограничение относительно другого элемента, то необходимо указать `id` этого элемента:



Здесь для текстового поля ввода `EditText` устанавливаются два ограничения относительно родительского контейнера `ConstraintLayout`, поэтому ограничения имеют значение `parent`, а сам `EditText` выравняется по левой и верхней границе контейнера. Верхняя граница кнопки `Button` также выравняется по верхней границе контейнера. А вот левая граница кнопки выравняется по правой границе `EditText`. Для этого в качестве значения атрибута передается `id` `EditText`:

`app:layout_constraintLeft_toRightOf="@id/editText"`

Позиционирование в центре

Если необходимо расположить элемент в центре контейнера по вертикали, то надо использовать пару атрибутов

`app:layout_constraintTop_toTopOf="parent"`

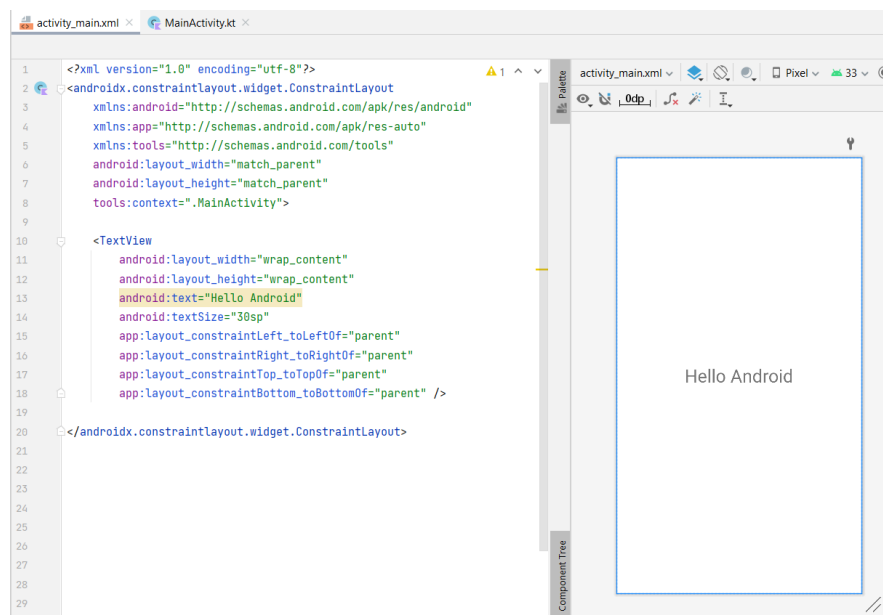
`app:layout_constraintBottom_toBottomOf="parent"`

Если необходимо расположить элемент в центре контейнера по горизонтали, то надо использовать следующую пару атрибутов

`app:layout_constraintLeft_toLeftOf="parent"`

`app:layout_constraintRight_toRightOf="parent"`

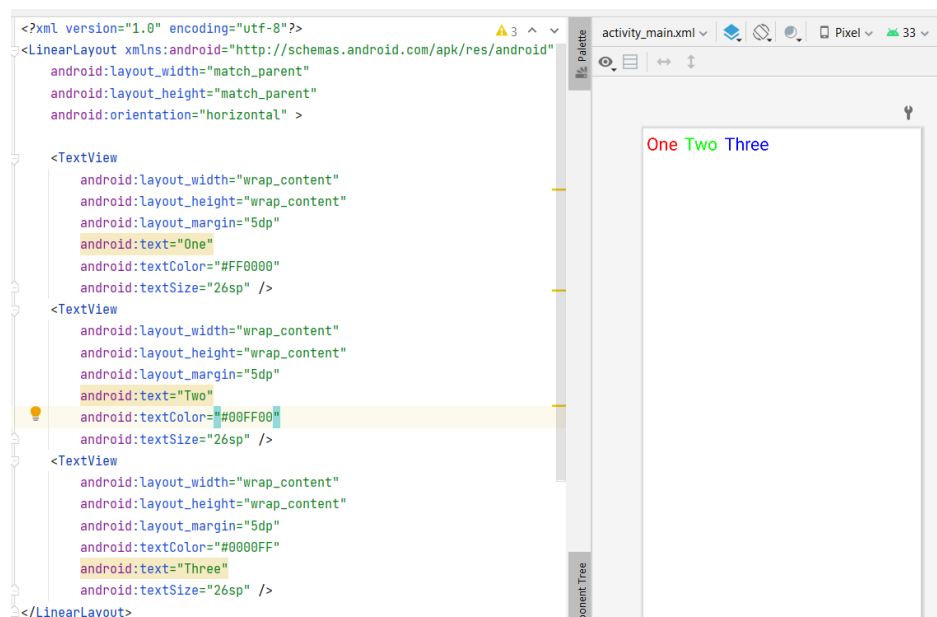
Соответственно для расположения в центре контейнера по вертикали и горизонтали надо применить все выше указанные четыре атрибута:



2. LinearLayout

Контейнер LinearLayout представляет простейший контейнер - объект ViewGroup, который упорядочивает все дочерние элементы в одном направлении: по горизонтали или по вертикали. Все элементы расположены один за другим. Направление разметки указывается с помощью атрибута android:orientation.

Если, например, ориентация разметки вертикальная (android:orientation="vertical"), то все элементы располагаются в столбик - по одному элементу на каждой строке. Если ориентация горизонтальная (android:orientation="horizontal"), то элементы располагаются в одну строку. Например, расположим элементы в горизонтальный ряд:



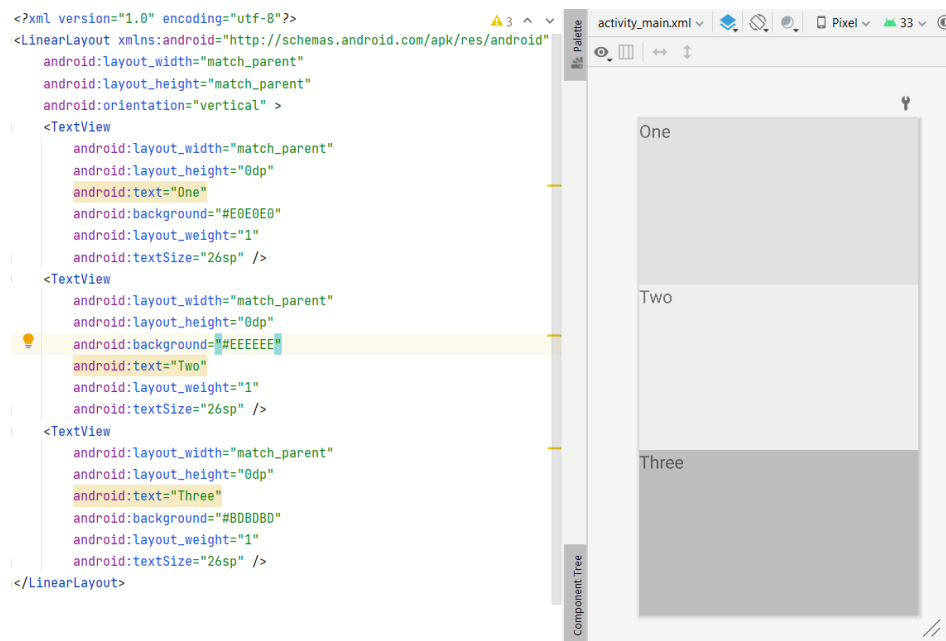
Если бы мы указали для LinearLayout атрибут `android:orientation="vertical"`, то элементы размещались бы по вертикали

Вес элемента

LinearLayout поддерживает такое свойство, как вес элемента, которое передается атрибутом `android:layout_weight`. Это свойство принимает значение, указывающее, какую часть оставшегося свободного места контейнера по отношению к другим объектам займет данный элемент. Например, если один элемент у нас будет иметь для свойства `android:layout_weight` значение 2, а другой - значение 1, то в сумме они дадут 3, поэтому первый элемент будет занимать $2/3$ оставшегося пространства, а второй - $1/3$.

Если все элементы имеют значение `android:layout_weight="1"`, то все эти элементы будут равномерно распределены по всей площади контейнера:

В данном случае LinearLayout имеет вертикальную ориентацию, поэтому все элементы будут располагаться сверху вниз. Все три элемента имеют значение `android:layout_weight="1"`, поэтому сумма весов всех элементов будет равна 3, а каждый элемент получит по трети пространства в LinearLayout:



При этом так как у нас вертикальный стек, то нам надо также установить для свойства `layout_height` значение `0dp`. Если бы LinearLayout имел горизонтальную ориентацию, то для свойства `layout_width` надо было бы установить значение `0dp`.

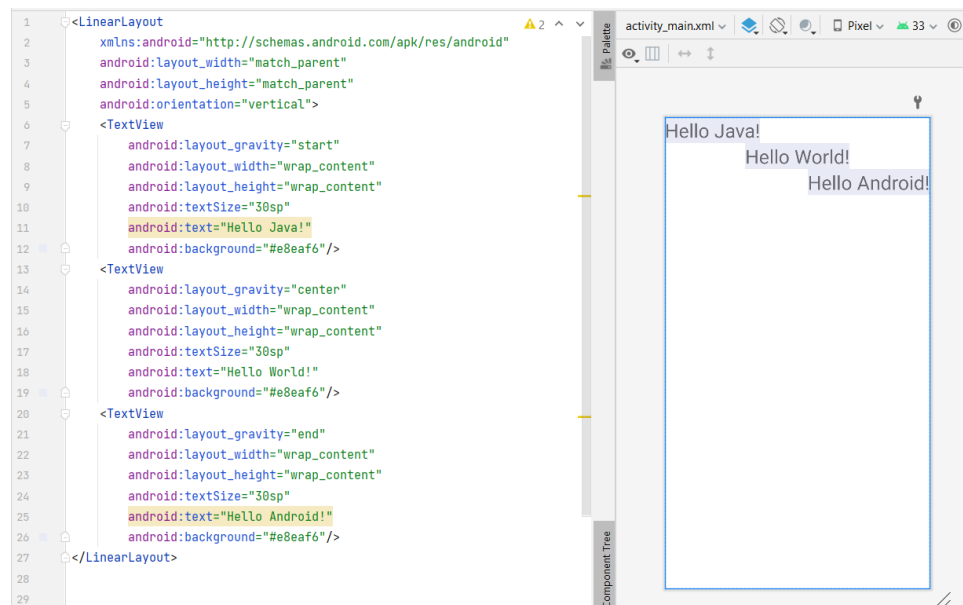
Layout_gravity

Атрибут `layout_gravity` позволяет устанавливать позиционирование относительно LinearLayout. Он принимает следующие значения:

- top: выравнивает элемент по верхней границе контейнера
- bottom: выравнивает элемент по нижней границе контейнера
- left: выравнивает элемент по левой границе контейнера

right: выравнивает элемент по правой границе контейнера
center_vertical: выравнивает элемент по центру по вертикали
center_horizontal: выравнивает элемент по центру по горизонтали
center: элемент позиционируется в центре
fill_vertical: элемент растягивается по вертикали
fill_horizontal: элемент растягивается по горизонтали
fill: элемент заполняет все пространство контейнера
clip_vertical: обрезает верхнюю и нижнюю границу элемента
clip_horizontal: обрезает правую и левую границу элемента
start: элемент позиционируется в начале (в верхнем левом углу) контейнера
end: элемент позиционируется в конце контейнера (в верхнем правом углу)

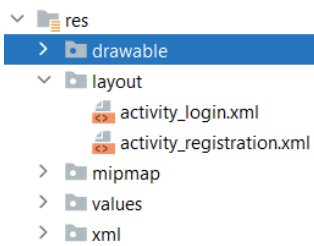
Например:



Стоит учитывать ориентацию контейнера. Например, при вертикальной ориентации все элементы будут представлять вертикальный стек, идущий сверху вниз. Поэтому значения, которые относятся к позиционированию элемента по вертикали (например, top или bottom) никак не будут влиять на элемент. Также при горизонтальной ориентации LinearLayout не окажут никакого влияния значения, которые позиционируют элемент по горизонтали, например, left и right.

Лабораторная работа 1.

Используя описанные элементы разметки (ConstraintLayout, LinearLayout, TextView, EditText, Button, CheckBox) создать 2 экрана xml разметки для регистрации и входа в систему пользователя. Расположение элементов, цвета, размеры и другие атрибуты для визуального представления экрана продумать самостоятельно. Примерный результат, который должен получиться:



Добро пожаловать!

Регистрация

По номеру

По email

Введите email

Введите пароль

Повторите пароль

ЗАРЕГИСТРИРОВАТЬСЯ

Добро пожаловать!

Вход

Введите email/пароль

Введите пароль

☐ Входить автоматически

ВОЙТИ